

5. Automatikus differenciálás

A numerikus algoritmusok gyakran támaszkodnak egy függvény deriváltjára. Az automatikus differenciálás (AD) olyan eljárás, amely a derivált **pontos** értékét (nem numerikus becslését) állítja elő közvetlenül a függvényt kiszámító programból.

Motiváció — a derivált előállításának módjai

1. „**Kézi**” deriválás papíron, majd a megfelelő szubrutin megírása.
2. **Numerikus deriválás**: a differenciahányadossal, $f'(x) \approx (f(x+h) - f(x))/h$.
3. **Szimbolikus deriválás** számítógépes algebrai rendszerrel (Maple, Mathematica, Reduce, Derive, ...).
4. **Automatikus differenciálás**.

A **numerikus** derivált előnye (+) és hátránya (-): + nincs előzetes munka a deriváltak előállítására, + akkor is működik, ha csak a kiszámító szubrutin adott, a képlet nem; – a **levágási hiba** miatt sok értékes jegy veszik el, és – a gyorsan változó deriváltakra alkalmatlan. A **kézi** deriválás előnye, hogy nincs levágási hiba és a gyorsan változó deriváltak is jól meghatározhatók; hátránya, hogy munkaigényes, hibaforrás, és csak képlettel adott függvényre alkalmazható.

Az AD mindezek előnyeit egyesíti: nem igényel előzetes ráfordítást, akkor is működik, ha csak a kiszámító szubrutin adott, **nincs levágási hiba**, a gyorsan változó deriváltakra is alkalmas, és műveletigénye általában kisebb a numerikus deriválásénál.

A módszer

Ha az $f(x)$ függvény képlettel megadható, illetve van őt kiszámító szubrutin, a derivált értéke a következő eljárással kapható:

1. Minden változó helyett használjunk **kételemű adatszerkezetet** (érték, derivált): az első a megszokott változóérték, a második egy derivált-érték.
2. Minden **változóra** a második (derivált) komponens kezdetben **1**, minden **konstansra** pedig **0**.
3. Ezután minden műveletre két szabály kell: egy az első komponensre (a szokásos művelet) és egy a második komponensre (a deriválási szabály szerint).

Példa: ha $f(x) = x_1 \cdot x_2$, akkor a szokásos programsor $F = X(1) \cdot X(2)$ helyett az AD a következőt számolja:

$$F(1) = X(1, 1) \cdot X(2, 1), \quad F(2) = X(1, 1) \cdot X(2, 2) + X(2, 1) \cdot X(1, 2).$$

Ez épp a szorzat deriválási szabálya — és sehol nem jelenik meg a deriváltfüggvény képlete.

Elemi differenciálási szabályok

A (érték, derivált) pár második komponensét az alábbi szabályok szerint frissítjük ($y = f(x)$):

- $a \pm x \rightarrow \pm 1$
- $a \cdot x \rightarrow a$
- $a / x \rightarrow -y/x$
- $\sqrt{x} \rightarrow 0,5/y$
- $\log(x) \rightarrow 1/x$
- $\exp(x) \rightarrow y$
- $\cos(x) \rightarrow -\sin(x)$

Végrehajtási módok és műveletigény

Az AD-nek két végrehajtási módja van:

- **Sima (forward) változat:** az alapfüggvény kiszámítási sorrendjét követi, az argumentumoktól halad a függvényérték felé; **egymenetes**.
- **Fordított (reverse) módszer:** előbb meghatározza a kiszámítási fát, majd fordított sorrendben, a redundanciák kihasználásával számol; **két menetet** igényel és nagyobb a tárigénye, cserébe a teljes gradiens nagyon olcsón adódik.

A műveletigényt az $L(\cdot)$ jelöli (az alapl műveletek $\{+, -, \cdot, /, \sqrt{\cdot}, \log, \exp, \sin, \cos\}$ feletti költség). A gradiensre például a sima változat $\leq 4nL(f)$, a fordított pedig már $\leq 4L(f)$ — utóbbi a változók n számától függetlenül. Ez teszi a fordított módot kulcsfontosságúvá sokváltozós feladatokban (és ez a mai gépi tanulás „visszaterjesztésének”, a backpropagationnek is az alapja).